Sanchamaan

30.7.27

Priority based Algo

Priority তে preemptive and non-preemptive 2 টাই আছে

Static and Dynamic

Static: Quest^n এ প্রশ্নতে যে value দেওয়া priority থাকবে ওটাই পরে থাকবে, no change

no change in priority

seenerio তে বলে দিবে preemptive নাকি non preemptive

Dynamic:

কোনো process যদি 2 momentum time এর বেশি waiting এ থাকে তখন তার priority +2 করে দিব

Quest^n এ seenerio তে বলা থাকবে যার burst time কম তাদের waiting time 2 এর বেশি হলে priority বাড়ায় দিবে

এটা দেখে বুঝব যে Dynamic

Preemptive এর ready queue তে আছে তখন process টা either ready queue তে গেছে বের হয়ে CPU থেকে or wait করছে,

Process	Arrival time	BT	Priority
P_1	2		High
P_2	0		Low

কোনটা আগে যাবে?

Ans: P_2, tho it's priority is low it came early

Non preemtive / Priority Based Algo

Process	AT	BT	Priority	CT	TAT	WT	RT
P_1	0	4	4	4	4	0	0
P_2	1	2	5	18	17	15	15
P_3	2	3	6	16	14	11	11
P_4	3	1	10 (Highest priority)	5	2	1	1
P_5	4	2	9	7	3	1	1
P_6	5	6	7	13	8	2	2

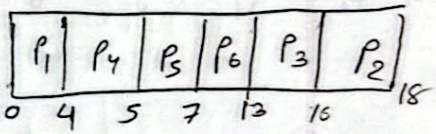
Quest^n এ যেমন priority কোনটার highest. Not always ascending

Ready Queue

at time 0 → P_1

" " 4 → P_2, P_3, P_4, P_5

" " 5 → P_2, P_3, P_5, P_6



Priority 4 এখন 3 বসাতে পারি

Response time = Waiting time

$$\text{Avg TAT} = \frac{4+17+14+2+3+8}{6} = \frac{48}{6} = 8$$

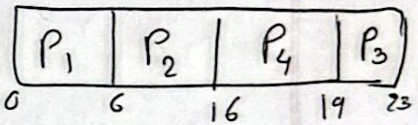
$$\text{Avg WT} = \frac{15+11+1+1+2}{6} = \frac{30}{6} = 5$$

Problem - 2

CT - AT TAT - BT

Process	AT	BT	Priority	CT	TAT	WT
P_1	0	6	2	6	6	0
P_2	4	10	1	16	12	2
P_3	4	4	2	23	19	15
P_4	8	3	1	19	7	4

Priority highest which is less



at 0 : P_1

at 6 : P_2, P_3

at 16 : P_3, P_4

at 19 : P_3

$$\text{Avg TAT} = \frac{48}{4} = 12$$

$$\text{Avg WT} = \frac{25}{4}$$

Sameka maan

10·8·26

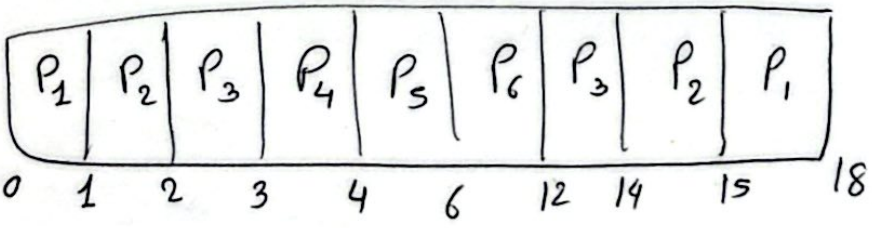
Priority

↳ Preemptive

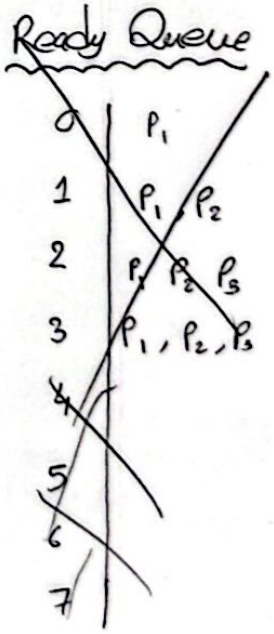
Process | Arrival Time | Burst Time | Priority

Process	AT	BT	Priority	CT	TT	WT
P_1	0	4	4	18	18	14
P_2	1	2	5	15	14	12
P_3	2	3	6	14	12	9
P_4	3	1	10(H)	4	1	0
P_5	4	2	9	6	2	0
P_6	5	6	7	12	7	1

10th number highest priority



Ready Queue



Scanned with CamScanner

Ready Queue

Time	Process
0	P_1
1	P_1, P_2
2	P_1, P_2, P_3
3	P_1, P_2, P_3, P_4
4	P_1, P_2, P_3, P_5
5	P_1 P_2 P_3 P_5 P_6
6	
7	

$P_1 \rightarrow 4$ 3

$P_2 \rightarrow 2$ 1

$P_3 \rightarrow 3$ 2

$P_4 \rightarrow 1$

$P_5 \rightarrow 2$ 1

$P_6 \rightarrow 1$

→ 2nd process stop,

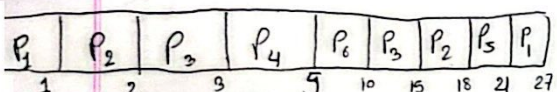
→ 2nd moment এ P_3 আসছে, queue তে আসছে P_1, P_2, P_3, এদের মধ্যে Highest priority P_3 এর তাই P_3 যাবে



Scanned with CamScanner

MP

Process	AT	BT	Priority	CT	TAT	WT
P_1	0	7	6	~~27~~ 18	~~18~~	~~14~~
P_2	1	4	4	~~18~~ 15	~~14~~	~~12~~
P_3	2	6	3	~~15~~	~~12~~	~~9~~
P_4	3	2	1 (H)	5	~~1~~	~~0~~
P_5	4	3	5	~~21~~	~~2~~	~~0~~
P_6	5	5	2	~~10~~ 11	~~7~~	~~1~~



Ready Queue

0: P_1 (7)

1: $P_1^{(6)}$, P_2 (4)

2: $P_1^{(5)}$, $P_2^{(3)}$, P_3 (6)

3: $P_1^{(6)}$, $P_2^{(3)}$, P_3 (5), P_4(2)

4: $P_1^{(6)}$, $P_2^{(3)}$, P_3 (5), P_4(1), P_5(3)

~~5: $P_1^{(6)}$, P_4, P_5, P_6~~

5: P_1(6), P_2(3), P_3(5), P_5(3)

Dynamic এর ক্ষেত্রে যত waiting time যত process update

Dynamic → Waiting → process → priority gets updated

Aging: যে process অপেক্ষা করে wait করছে তাদের priority বাড়ানো

Exam ★★

Ques

3 momentum এর পরের থেক যেসব process wait করছে তাদের priority এক করে বাড়ানো

Practise this

⇒ আর যদি quest এ এরকম কোনো কিছু না বলে তখন static

⇒ দুইটা process এর priority same হলে AT দেখব

Dynamic Priority ★★ Practise

Last year final/mid quest

Round Robin

Process

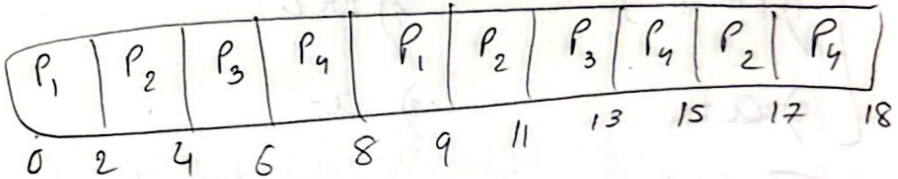
Till now যত alg করলাম তাদের মূলত practicality র অভাব ছিল। process শুরু হওয়ার time এ তো বলতে পারবনা যে process টা কখন শেষ হবে Round Robin এটা fix করে।

Time sharing: multiprogramming এ অনেকগুলো program একসাথে run করলে, কোন program কখন end করবে তোমার জন্য Round Robin

Quantum time: কতক্ষণ পর CPU থেকে বের হবে

⇒ Round Robin এ quantum time থাকবে সেই time পর্যন্ত process গুলো switching হবে

Process	AT	BT	CT	TAT	WT	Response
P_1	0	3	9	9	6	0
P_2	0	6	17	17	11	2
P_3	0	4	13	13		4
P_4	0	5	18	18	13	6



Ready Queue

0	P_1 P_2 P_3 P_4
2	P_2 P_3 P_4 P_1
4	P_3 P_4 P_1 P_2

$P_1 \rightarrow 3\ 1$

$P_2 \rightarrow 6\ 4\ 2$

$P_3 \rightarrow 4\ 2$

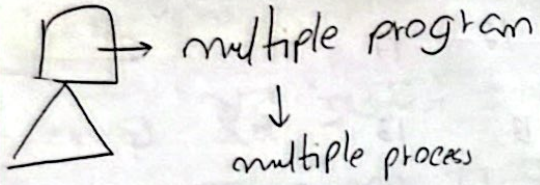
$P_4 \rightarrow 5\ 3\ 1$

AT wise যাবে

Sancha maan

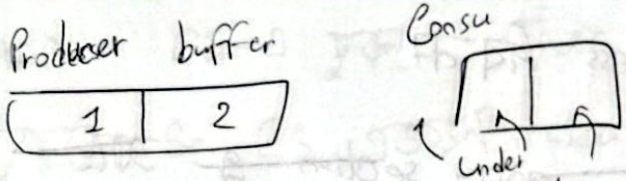
13.8.26

Process synchronization



	P_1 Producer	P_2 Consumer
ES	1) int a = 1	1) int a
	2) a++	2) a--
CS	3) print a	3) save buffer
	if a=2 printer → print → RM	

একাধিক process একটা shared buffer use করে এবং সেটা updated হয়ে যায় বলে আগা outcome দিতে পারছেনা



⇒ Producer consumer problem → process synchronization এর কারণে issue

Process synch এর workflow কে 4 ভাগে ভাগ করা যায়

1) Entry section (ES)
2) Critical section (CS)
3) Exit section (ExS)
4) Remainder section (RS)

Property

1) Mutual exclusion

CS এ একাধিক process যায় CS portion এ একটা process কেই access দিবে একটা process থাকাকালীন অন্য কোনো process

~~P_1 P_2 P_3 P_4~~

কে access দিবে না

~~P_3 P_4 entry section এ আছে~~

P_1 P_2 P_3 P_4 ৪ টা process আছে

P_1 P_2 in RH sect^n

P_3 P_4 in entry sect^n, যারা entry করতে চাইবে শুধু তাদেরকে নিয়ে ভাবব

Progress - Available process থেকে যারা CS এ entry চাইবে শুধু তাদের নিয়ে ভাবব

Bounded waiting

P_3 যাওয়ার পরে আর 3 টা process CS এ ঢুকবে then P_3 আবার ঢুকবে, So CS এতক্ষণ waiting এ ছিল ~~তাই~~ এই waiting টা select করা যায়

Mutual exclusion algorithm

Peterson's Algorithm — Software based

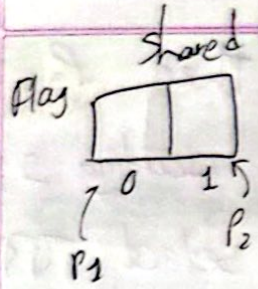
P_1

```
do{ flag[0] = true;
    turn = 1
while (flag[1] && turn==1);
  // CS
  flag[0] = false;
  // RM
} while(true)
```

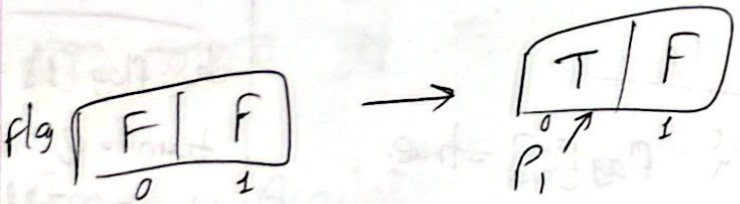
P_2

```
do{ flag[1] = true;
  turn = 0;
while (flag[0] && turn==0);
  // CS
  flag[1] = false
  // RM
} while(true);
```

Scanned with CamScanner



P_1 এর কাজ 0 নং index নিয়ে flag initially false থাকবে। P_1 নেবে then true হবে



P_1 এর while loop এ

flag[1] = false and turn = 1

And condit false so CS এ ঢুকবে

Then Flag এর ~~value~~ value থাকবে

false থাকবে then remainder এ চলে যাবে।

Turn make sure করে opposite process টা CS এ আছে কি না

P_2 এর ক্ষেত্রে

flag at index 1 কে true করবে

~~while~~ এরপর flag[0] = false and turn = 0

So Condit false এবার CS এ যাবে

CS এ P_2 আছে এবার যদি P_1 চায় CS এ যেতে, flag এর 0 index এবার false থাকবে, তা যেতে চাইলে

true হবে then turn = 1

Condit" true তাহলে এরা while loop এই আটকায় যাবে

concurrently while loop এ check করতেই থাকে, যদি condit" false হয় cs এ ঢুকে যাবে

Drawbacks of Peterson

1) while loop এ আটকায় যায়, Computat" power use

2) Multitasking হচ্ছে না, P_1, P_2 ছাড়া অন্য কোনো process নিয়ে কাজ করতে পারবেনা → flag এর size 2

3) দুইটা shared buffer use করছে - turn, flag

⇒ Peterson এর issue কে solve করতে

Test and set algorithm

Hard ware based

It can work with more than 1 process

```
lock = false
boolean test_and_set (boolean * target)
{
    boolean rv = *target;
    target = true;
    return rv;
}

do {
  while (test_and_set (& lock)
      // do nothing
      // cs
  lock = false;
      // RM
} while (true)
```

Lock = target memory address pass হচ্ছে

do{ -- যেকোন code হতে

lock এর value initially false.

lock এর address passed to target

target এর value রাখা হচ্ছে

then target = true

so lock এর মান হচ্ছে true

return মান

false

while condit false so হচ্ছে

CS এ যাবে

এখন অন্য কোনো process যদি আসতে

চায় while এ ঘুরতে থাকবে

Samsha maam

17.8.26

Compare and swap (Hardware based)

flock = 0 By default

```
int compare_and_swap (int *value, int expected,
                      int new_value)
{
    int temp = *value
    if (*value == expected)
        *value = new_value;
    return temp;
}
```

এখান থেকে execution হবে

```
do {
    while (compare_and_swap (flock, 0, 1) != 0)
        /* CS */
    lock = 0
        /* RM */
} while (true);
```

1 != 0

0 != 0

false হলে CS এ ঢুকবে

True হলে loop এর মধ্যে আটকায় যাবে

Mutex lock

- acquire()
- release()

available = true;

```
acquire() {
while (!available) {
   /* do nothing */
   available = false;
}
release() {
available = true;
}
```

```
do {
acquire()
// cs
release()
// RM
} while (true)
```

কোনো একটা process P1 → CS এ যেতে চাচ্ছে, P1 call করবে acquire() funct" কে.
This funct" checks availability
In acquire funct" we check
if while (!available) (True) — available initially true
তখন loop এ ঢুকে
available = false
at the same moment another process P2, CS এ যেতে চাইল. সে acq()
func call করবে in while loop
Check available এসে false এ
আছে so while loop এ ঘুরতেই থাকবে। এমন P1 কাজ শেষে release(

Scanned with CamScanner

finish call করবে তখন available = true হবে। P_2 যে এতক্ষন while loop এ ঘুরছিল যখন available = true তখন তখন while loop এ ভিতরে ঢুকবে that is CS এ ঢুকতে পারবে

Deadlock: while loop এ প্রত্যেকবার পড়ে যাওয়া

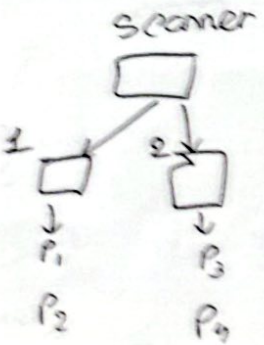
কোনো একটা resource কে পাওয়ার জন্য অনেক process Busy Waiting এ থেকে আগে বারবার while loop check করবে এটা deadlock situation create করে।

এতক্ষন যা করলাম

একটা particular resource use করছিলাম সেই resource এ একটা করে process কে রাখছিলাম, But এটা আমরা চাই Resource management. CS এ কয়েকটা process একসাথে থাকতে চায়

Scenario

একটা office এ 2 টা scanner. আমরা চাচ্ছি at a time 2 টা scanner একসাথে use হোক। 2 টা scanner parallely একটা করে process scan করতে পারে



আগে mutual exclusn একটা process নিয়েই কাজ করে CS এ, এখন আমরা চাই scaner এ multiple process যাবে, এটা achieve করি Semaphore দিয়ে

⇒ একটা resource এর instance যেখানে একটা process ই যাচ্ছে, কাজ করে release করে দিচ্ছে

এখন per instance → একটাই process allowed

S = semaphore এর value
= instance এর number

দুইটা scaner so S=2

Semaphore

```
wait (s) {                          // 2 <= 0 ? false
while (s <= 0) {
  // * busy waiting *//
  s--;
}
signal (s) {                        // 0
  s++;                              // 0+1
}                                   // s = 1
do {
  wait (s);
  // CS //
  Signal (s);
  // RM //
} While (true);
```

P1 এখন কেস করে signal() call করবে

so এখন P3 process এ ঢুকতে পারবে

execution starts →

so loop এর ভিতর ঢুকবে P1 process একটা resource ফেল এই moment এ P2 call করবে wait then s=1 While (1 <= 0 ?) false so loop এ ঢুকবে

Now P3 called Wait() এখন s = 0 While (0 <= 0 ?) true so While loop এ ঘুরতেই থাকবে

Scanned with CamScanner

⇒ Semaphore is the best solut^{n} out of all mutual

Busy waiting: While (s<=0)

Issues of Semaphore

1) deadlock situat^{n}
2) busy waiting

Semaphore 2 types

1) Binary → যে semaphore এর value 0, 1
2) Counting
   যে semaphore এর value
   এক এর অধিক

scenario থেকে বুঝতে হবে instance কয়টা দিবে, directly বলা থাকবেনা

যদি counting scenario বলা থাকে s= more than 1 তবে নিয়ম করার maths.

☆ Scenario

→ P_1, P_2 দুইটা process আছে, আমরা বলছি P_2 ☆ first execute হবে, আগে cs এ যাবে

P_1 cs এ যাওয়ার try করলেও পারবেনা বুঝতে, Main concern P_2 আগে যাবে

P_2 শেষ হলে P_1 যাবে

s = 0 initially

wait() → s--

signal() → s++

দুইটা process এর জন্য আলাদা signal call করা লাগবে

P_2 এর জন্য signal() call করব যেন s++ হয় so এখন P_2 ঢুকবে,

Scanned with CamScanner

P_1 এর জন্য wait() call করায় যেন s-- হয় তাহলে তা ঢুকতে পারবেনা

Learning: different process different function call করতে পারে

Semaphore maam

28.8.26

wait( ){
s→value--;
if (s→value<0){
add process to
}

while এর problem solve করতে

list থেকে যেন কোন একটা process wakeup করবে

```
type struct{
 int value;
struct process list;
} semaphore;
```

list এ রাখার আগে আগে block করে দিচ্ছি

```
wait(semaphore *s){
 s→value--
if (s→value<0){
add process to
 s→list
}
 block(p);
```

```
signal(semaphore *s){
s→value++
if(s→value<=0){
remove s from
list;
}
wakeup(p);
}
```

```
do{ wait(); //CS
    signal() //RM
} while(true)
```

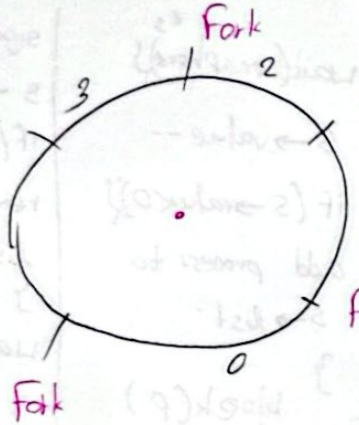
while loop এর problem solve করতে list এর আরেক process তুলে দিচ্ছি

Scanned with CamScanner

Dining philosopher problem

⇒ dinner arrangement
⇒ 5 philosopher
⇒ 5 table arranged
⇒ Fork, spoon must

⇒ Right hand, left hand both use
⇒ Then spoon এর shortage হবে



⇒ 5 টা fork/spoon দিয়ে both hand use করা যাবেনা,
⇒ 10 টা লাগবে LHS, RHS এ একসাথে লাগবে তবে একটা LHS / RHS এ পাবে

This is the issue of bounded buffe

Solution

1) 5 জনের মধ্যে একজনকে বাদ দিলে at least দুইজন পাবে বাকি দুইজন wait করবে

এই solution টা যদি semaphore দিয়ে করি

```
wait ( fork [i])
wait ( fork [i+1] % n)   ← total no. of person
 // eat
signal (fork [i])
signal ( fork [i+1 % n])
```

Scanned with CamScanner

Another solution

2) Odd no. person RHS থেকে শুরু করবে
Even " " LHS " " "

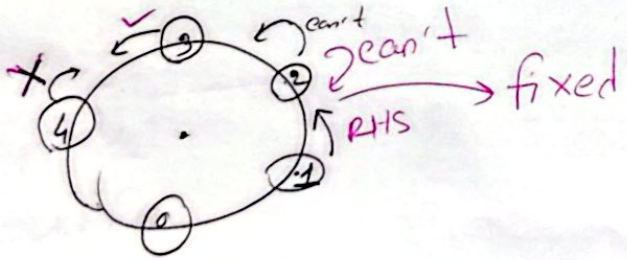
দুইটা solution এই কাউকে wait করাই লাগবে

একইসাথে না, either way

Let 1 starts from Right

3 → Right

Even number person তখনি বসতে পারবে, একইসাথে হচ্ছেনা
odd start করলে even waits



buffer share:

wait(fork[i])	wait (fork[i+1%n])
wait (fork[i+1%n])	wait (fork[i])
//eat	//eat
signal (fork[i])	signal (fork[i+1%n])
signal (fork[i+1%n])	signal (for[i])

Scanned with CamScanner

magnetic disk

1) No. of sector in disk = 2 × no. of plate × no. of track × no. of sector per surface

2) Address size = $\log_2$ (no. of sector in disk) – bits

3) sector capacity = 2 × no. of plates × no. of track per surface × no. of sector per surface × 1 sector capacity tracks

4) Disk Access time = sick time + rotational latency + 1 sector transfer time + additional latency delay

5) avg rotational latency = $\frac{1 \text{ disk rotation}}{2}$

sick time
disk ঘুরানোর time

6) sector transfer time = $\frac{1 \text{ disk rotation time}}{\text{no. of sector per track}}$

⇒ একটা file এর অনেক info/ attribute থাকে যেমন
file name, size, data, security, details

file system
operating system এর file system NTFS
swap → mid term sch এর জন্য swap space maintain করে

File directory (F.D)
Single F.D: same name 1 2 টা file থাকে না
directory তে রাখতে পারবেন
single user only

Two level F.D

solves the problems of single F.D

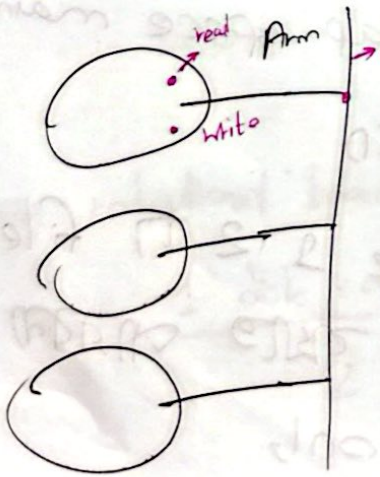
⇒ IOS এর ক্ষেত্রে root directory তে কিছু রাখতে পারিনা

⇒ Android এ পারি, changes করতে পারি



Disk type

SSD তে file save হয়

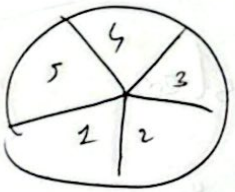


যে disk দুইপাশে ফিরতে পারি magnetic disk

এক পাশে ফিরতি optical disk

⇒ Arm এর thru তে disk rotate করে read / write

⇒ Why spindler



⇒ 36 টা track

⇒ 5 টা sector

⇒ Formula written earlier

MP

1) Disk এ 17 টা plate, 2 টা
2) surface per plate
3) 3k track per suface
4) 4K sectors per "
5) 3.48 bytes per sector
6) Disk rotation 9000 rpm

7) Sick time 15 ms

Scanned with CamScanner

Unit Conversion করতে হবে পারো

Secta cap = 2 × 17 × 3 × 4

What is file system

Extens" of " "

Disk এর structure → V.V.I

disk math

formula

1 ti set

Compulsory

mixed

Last yr final

1st Chp সব imp

2nd " IPC

Scanned with CamScanner